Reduce Detection Latency of YOLOv5 to Prevent Real-Time Tracking Failures for Lightweight Robots

Jianxin Xue
School of Computer and Information Engineering, Institute for Artificial Intelligence, Shanghai Polytechnic University
Shanghai, China
jxxue@sspu.edu.cn

Husheng Chen
School of Computer and Information Engineering, Institute for Artificial Intelligence, Shanghai Polytechnic University
Shanghai, China
20211510161@stu.sspu.edu.cn

Yaohua Hu
School of Computer and Information Engineering, Institute for Artificial Intelligence, Shanghai Polytechnic University
Shanghai, China
20191113606@stu.sspu.edu.cn

Minyu Chen
Shanghai Jiao Tong University
Shanghai, China
minkow@sjtu.edu.cn

Ling-I Wu
Shanghai Jiao Tong University
Shanghai, China
edithwuly@sjtu.edu.cn

Xi Chang
School of Computer and Information Engineering, Institute for Artificial Intelligence, Shanghai Polytechnic University
Shanghai, China
changxi@sspu.edu.cn

ABSTRACT

Lightweight robots are frequently engaged in real-time tracking tasks to provide human companionship services. For effective target tracking, the YOLO series is often employed as a lightweight object detection framework in robot systems. However, YOLO still demands substantial resources to train larger-scale models, striking a balance between accuracy and resource efficiency. Deploying YOLO directly on robots with limited computing resources can lead to significant delays in detection, compromising the effectiveness of tracking tasks. A deeper concern arises from the prevalent use of CPUs as the primary computing units in robots, rendering many existing model optimization techniques, which primarily target GPU computing, unsuitable for this context.

To tackle this challenge, we propose a novel detection framework called SLCNet-YOLOv5, specifically designed for deployment in CPU-centric computing environments on robots. The core concept of SLCNet-YOLOv5 entails substituting the native YOLOv5 backbone network with SLCNet, which is a simplified version derived from the existing CPU convolutional neural network, PP-LCNet. It is important to note that our aim is not to enhance PP-LCNet to improve inference accuracy but rather to simplify it to enhance inference speed, while tolerating a certain degree of accuracy loss. This is because excessive inference latency may lead to real-time tracking failures. By employing a backbone network optimized for CPU-centric computation and reducing the computational complexity of the detection model, SLCNet markedly reduces latency, expediting the detection process, with only a minor trade-off in accuracy. In comparison to the performance of the state-of-the-art detector YOLOv5, experimental results on publicly available coco-foot-and-leg and PASCAL VOC datasets demonstrate significant enhancements in detection speed per image on CPU-centric terminals, with respective increases of 62.8% and 81.3%, alongside marginal declines in mean Average Precision (mAP) at 0.5 Intersection over Union (IoU) threshold, with losses of 0.077 and 0.165.

KEYWORDS

Real-Time Tracking, Object Detection, Inference Latency, Lightweight Robots

ACM Reference Format:
Jianxin Xue, Husheng Chen, Yaohua Hu, Minyu Chen, Ling-I Wu, and Xi Chang. 2024. Reduce Detection Latency of YOLOv5 to Prevent Real-Time Tracking Failures for Lightweight Robots. In *15th Asia-Pacific Symposium on Internetware (Internetware 2024), July 24–26, 2024, Macau, China.* ACM, New York, NY, USA, 10 pages. https://doi.org/10.1145/3671016.3671392

1 INTRODUCTION

Lightweight robots play crucial roles in various domains, acting as caregivers for guiding and accompanying patients [8, 40], pathfinders assisting travelers during boarding procedures, and trackers or followers in dynamic environments [2, 20]. Their utility in aiding humans is transforming practices within these sectors. Real-time target tracking stands as a foundational task for these robots to achieve proficiency.

Unlike their heavier counterparts, lightweight robots like NAO (Aldebaran Robotics) and Pepper (SoftBank Robotics) rely on lightweight real-time object detection frameworks such as YOLO [1, 24, 33–35, 38] instead of resource-intensive large-scale neural networks. Notably, the YOLO framework has evolved through several iterations[1, 24, 34, 35, 38, 43]. In this study, we adopt YOLOv5 [7] as our object detector due to its efficacy in tasks involving target

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Internetware 2024, July 24–26, 2024, Macau, China
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-0705-6/24/07
https://doi.org/10.1145/3671016.3671392

tracking, particularly human tracking, on lightweight robotic platforms. YOLOv5 strikes a favorable balance between accuracy and computational resources, making it well-suited for deployment in such scenarios.

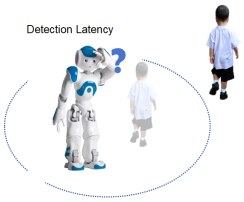


Figure 1: Tracing failure scenario of lightweight robots

Despite the availability of suitable detection frameworks implemented on low-computational-resource platforms, the inherent delay in detection often adversely affects tracking performance. For instance, as illustrated in Figure 1, upon initially surveying its surroundings, the robot successfully detects the target (a child) and subsequently determines its position. However, due to the latency intrinsic to the detection process, the child moves beyond the detection range, leading to a failure in tracking by the robot.

In contrast to prior studies [7, 16, 26, 45], our focus lies not on prioritizing accuracy improvements under resource constraints as the primary objective, but rather on adopting a more positive perspective that *emphasizes speed enhancement within an acceptable range of accuracy loss*. We advocate for reducing the burden on the detection model while still maintaining an acceptable level of accuracy in order to achieve speed improvements, rather than pursuing accuracy enhancements at the expense of high computational costs. This approach proves particularly effective for tracking tasks, where timely detection is crucial to prevent tracked targets from exceeding the detectable range.

Aligned with the aforementioned perspective, we present a novel detection framework called SLCNet-YOLOv5, specifically designed for deployment in CPU-centric computing environments on robotic platforms. The core technical aspects of SLCNet-YOLOv5 encompass two key points: 1) Seamless integration of PP-LCNet [4], existing CPU convolutional neural networks (CNNs), in lieu of the native YOLOv5 backbone network. This decision arises from the recognition that detection models trained with PP-LCNet as backbone networks adeptly leverage the CPU's processing capabilities, adept at handling complex control logic and serial tasks. 2) SLCNet represents a simplified variant of PP-LCNet. This involves simplifying PP-LCNet by reducing the complexity of the activation function and downsizing the convolutional kernel in the tail of networks. Such refinement is meticulously executed to ensure slight impact on the model's accuracy. This initiative is geared towards further enhancing the framework's speed performance.

Experimental results on publicly available datasets demonstrate a significant reduction in latency compared to the native YOLOv5. SLCNet-YOLOv5 exhibits superior speed, surpassing native YOLOv5 by up to 62.8% on the Coco-foot-and-leg dataset and 81.3% on the PASCAL VOC dataset. Regarding detection accuracy, SLCNet-YOLOv5 incurs a marginal loss in mean Average Precision (mAP) at 0.5 Intersection over Union (IoU) threshold, with losses of 0.077 and 0.165, respectively. These notable improvements in speed, accompanied by a minor decrease in accuracy, affirm that SLCNet-YOLOv5 demonstrates promising performance aligned with our proposed perspective.

The contributions of this paper are summarized as follows:

- We propose a novel object detection framework, SLCNet-YOLOv5, which replaces the backbone network of YOLOv5 with SLCNet, thereby significantly improving real-time object detection speed while maintaining acceptable accuracy levels for CPU-centric terminals. This enhancement proves beneficial in preventing tracking failures on lightweight robots.
- We have designed an SLCNet backbone network by simplifying the PP LCNet architecture, which is specifically tailored for tasks with inference speed priority and accuracy considerations on CPU-centric devices.
- We implement the SLCNet-YOLOv5 object detector, resulting in a notable improvement in speed while maintaining acceptable detection accuracy levels on datasets such as Coco-foot-and-leg and PASCAL VOC.

2 PRELIMINARY

This section introduces YOLOv5 and PP-LCNet and analyze their advantages and limitations.

2.1 YOLOv5

We have chosen YOLOv5 as the object detection framework for this study based on two principal considerations. Firstly, YOLOv5 demonstrates advancements in accuracy compared to previous iterations, such as YOLOv4. Moreover, its design incorporates lightweight model features, facilitating a reduction in model size and computational resource requirements while maintaining high performance. Secondly, recent developments, like YOLOV [38] and YOLOv7 [43], prioritize enhancing inference accuracy. Our initial investigation, conducted on the coco-foot-and-leg dataset (as referenced in Section 4.2) of YOLOv7, revealed significantly increased inference parameters (5.18 times), layer counts (1.95 times), and CPU-centric inference time (10 times) compared to YOLOv5. Consequently, the larger model size and slower inference speed rendered them unsuitable as detection models for lightweight robots. Thus, YOLOv5 emerges as an optimal choice for real-time object detection on computing lightweight robots.

YOLOv5 is a single-stage object detection framework. Figure 2 shows the framework of the YOLOv5s. The **Input** segment of YOLOv5 showcases adaptability by accommodating various input sizes, thereby enhancing the model's versatility. Leveraging data normalization and augmentation techniques further bolsters accuracy and enriches the quality of the training data. The **Backbone** and **Nest** segments are used to extract the features of input imagines and enhance the representation power of feature model to better handle targets of different scales and complexities. The **Head** segment, comprising multiple detection layers, facilitates multi-scale

detection capabilities, enabling the model to discern targets of varying sizes. Additionally, it incorporates non-maximum suppression to mitigate redundant detection outcomes.

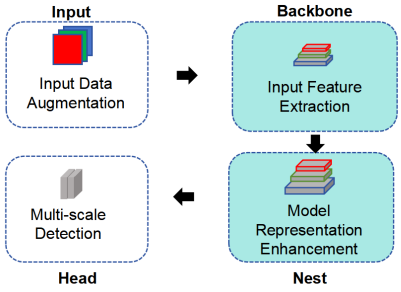


Figure 2: Framework of YOLOv5.

Based on our investigation, it is evident that the backbone network accounts for an average detection time consumption of 62%, significantly surpassing the combined time consumption of the other three segments. Hence, the decision to replace the backbone network while retaining other components emerges as a prudent choice.

2.2 PP-LCNet

PPLCNet is a lightweight network that leverages IntelCPU endside inference features. It incorporates several key operations: some Depthwise Separable Convolution (DSC) operations, squeeze and excitation operations in SqueezeandExcitation (SE) modules. The network structure of PP-LCNet is shown in Figure 3.

The DSC operation disassembles conventional convolution processes into two sequential steps: depthwise convolution (DW) and pointwise convolution (PW). In the depthwise step, spatial filters are applied individually to each channel via 3x3 convolutions, followed by the pointwise step where outputs from different channels are merged. After both of the two steps, an H-switch activation function is applied to activate feature maps as non-linear transformation. It is defined as follows:

$$\text{H-swish}[x] = x \times \frac{\text{ReLU6}(x+3)}{6} \tag{1}$$

H-swish function was initially introduced in MobileNetV3 [18]. Prior to the introduction of the H-swish activation function, the ReLU activation function was fast but faced the issue of neurons stopping outputting values other than 0 during training, leading to a network output of 0. The Swish activation function played a significant role in optimization and generalization, but it was computationally intensive. The adoption of the H-switch activation function offers a tradeoff between speed and accuracy.

The depthwise and pointwise convolution are respectively responsible for performing convolution operations on the features of each channel and inter-channels, while the activation function is responsible for non-linear transformation of the output of each channel or inter-channels. The different feature map output spatial dimensions after each stage's (shown below Figure 3) are set to capture features of different scales at different network levels. Together, these convolution operations and activation function promote the improvement of feature extraction and expression capabilities of neural networks.

To improve the detection model's accuracy, the SE module is integrated at the end of the network. After the Global Average Pooling (GAP) operation extracts global information, a 1280-dimensional Fully Connected (FC) layer enhances the model's fitting accuracy. The final feature vector is then mapped to the output class space through a 1000-dimensional FC layer. The SE module collaboratively integrates post the final 5x5 DSC to further refine accuracy.

This paper delves into the object detection capabilities of lightweight robots. However, the constrained computing power of these robots not only impedes the adoption of high-performance, large-scale visual inference neural networks for object detection tasks but also complicates the utilization of native YOLOv5. This complication arises from YOLOv5's emphasis on optimizing GPU inference, which contrasts with the CPU inference commonly employed by lightweight robots. Following extensive experimentation, the decision was made to substitute YOLOv5's backbone network with PP-LCNet. This strategic shift aims to implement an efficient object detection network within lightweight robot systems, effectively addressing object tracking tasks.

3 APPROACH

In this section, we propose a **S**implified PP-**LCNet** (SLCNet) as the backbone network of YOLOv5. The original backbone network utilized by YOLOv5 was CSPDarknet53, a variant of the Darknet backbone network [35]. CSPDarknet53 typically exhibits superior performance on GPUs due to its optimized parallel computing capabilities. This complexity may lead to faster detection speeds specifically on GPUs, instead of CPUs. Hence, there arises a necessity to replace the backbone network in YOLOv5 with one tailored for CPU computing. In the subsequent subsections, we first outline the design of our SLCNet and then provide further details on SLCNet-YOLOv5.

3.1 Simplified PP-LCNet

In the original PP-LCNet structure, the depthwise step within a DSC operation captures inter-pixel relationships in a channel, while the pointwise step addresses inter-channel relationships. The adoption of the H-swish activation function after both depthwise and pointwise convolutions aims to introduce nonlinearity, thereby enhancing the network's expressive capacity[5].

To streamline the detection model, we propose a simplification strategy that *retaining the application of the H-switch activation function solely after the depthwise convolution, while removing it after the depthwise convolution.* This strategic motivation comes from two aspects, by analyzing the workflow of a DSC operation shown in Figure 4. 1) In the depthwise dimension, within the same channel, there exist numerous intricate nonlinear characteristics among pixels. Neglecting these nonlinear traits could significantly diminish the model's expressive capacity, thereby impeding precise object detection. Preliminary research has found that the accuracy loss is as high as 22.3%. 2) Capturing the non-linear features in the channel dimension, it not only increases computational overhead but also may lead to loss of information between RGB channels[37]. Thus, we prefer to sacrifice non-linear features in the channel dimension.

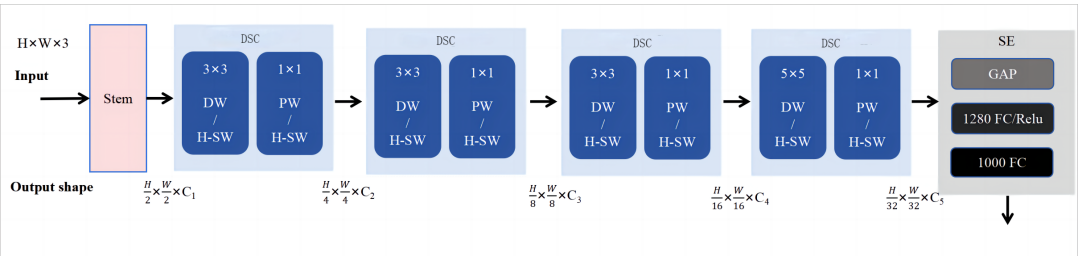


Figure 3: Architecture of PP-LCNet.

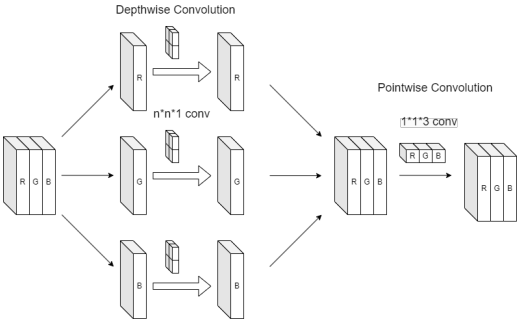


Figure 4: Workflow of a DSC operation

Empirical evidence in Section 5.1 shows that this simplification strategy yields detection models with sufficient expressive power to handle object tracking tasks.

Furthermore, the original PP-LCNet leverages four sequential convolution kernels to extract sufficient features. Being aware that stacked sequential convolution kernels can achieve a similar receptive field compared with larger convolution kernels, we point out that the shape of the last 5x5 convolution kernels can be reduced considering the depth of the CNNs. Thus, We adopt another simplification strategy that *set the DSC convolution kernels in the tail of the network to 3x3 convolution kernels*. This modification on convolution kernels contributes significantly to reducing the size of convolution kernels while preserving the key neighbor receptive field for downstream tasks.

Subsequent experimentation demonstrates that the SLCNet which adopts the two strategies yields additional computational speed-up while only marginally sacrificing accuracy.

3.2 SLCNet-YOLOv5 Framework

In the context of resource restricted robots, we've opted to YOLOv5s [47] as our detection infra-structure. YOLOv5s [47] is a member of the YOLOv5 family, characterized by different network scales including small (s), middle (m), large (l), and extra-large (x). YOLOv5s is distinguished by its reduced parameter count and layer complexity, resulting in expedited inference times compared to its counterparts. Conversely, YOLOv5x boasts increased parameters and layers, enhancing inference accuracy.

In addition, considering the CPU-centric computing characteristics and the need for tracking tasks (mentioned in Section 3), we've substituted the backbone network of YOLOv5 with SLCNet. This substitution entails integrating SLCNet into the YOLOv5 framework shown in Figure 5 and several considerations have been taken into account:

- Input Size and Preprocessing: YOLOv5 typically operates on images of size 640 × 640 × 3. To enhance model robustness, we employ data augmentation techniques such as mosaic, brightness, contrast, and saturation adjustments. These methods diversify the training images, thereby improving the model's generalization ability.
- Hyperparameter Adjustment: To accommodate the substitution of the backbone network and optimize model performance, several hyperparameters are adjusted. we adopt the Warmup strategy proposed by Xiong et al. [48] to gradually adjust hyperparameters, including learning rate, momentum, and bias learning rate, during the initial stages of training. This approach helps stabilize the model by gradually transitioning to the designated learning rate. In our study, the batch size is set to 68, the learning rate is adjusted to 0.01, the momentum parameter is configured to 0.937, and the weight decay factor is set to 0.0005.

During SLCNet-YOLOv5s training, the backbone network progressively learns diverse scales and abstract feature representations. This enables effective image data representation, facilitating object recognition across various categories during detection. Specifically, the initial Stem convolution applies a 3x3 depth separable convolution with a stride of 1, increasing input dimensions from 16 to 32. The first DSC stage processes 32-dimensional input feature maps, producing 64-dimensional output maps via a 3x3 depthwise convolution with a stride of 2, followed by a pointwise convolution. Subsequent stages follow a similar pattern: the second stage handles 64-dimensional input to generate 128-dimensional output, the third processes 128-dimensional input to yield 256-dimensional output, and the fourth converts 256-dimensional input to 512-dimensional output using depthwise and pointwise convolutions with appropriate strides.

The SLCNet-YOLOv5s network processes information in a step-by-step manner, which aligns well with the sequential processing capabilities of CPUs. Unlike GPUs, which excel in parallel computation, CPUs are optimized for handling tasks sequentially. By employing multiple stages of convolution, the network can capture intricate patterns and features at various levels of abstraction. This enables more comprehensive analysis of the input data, leading to improved detection accuracy.

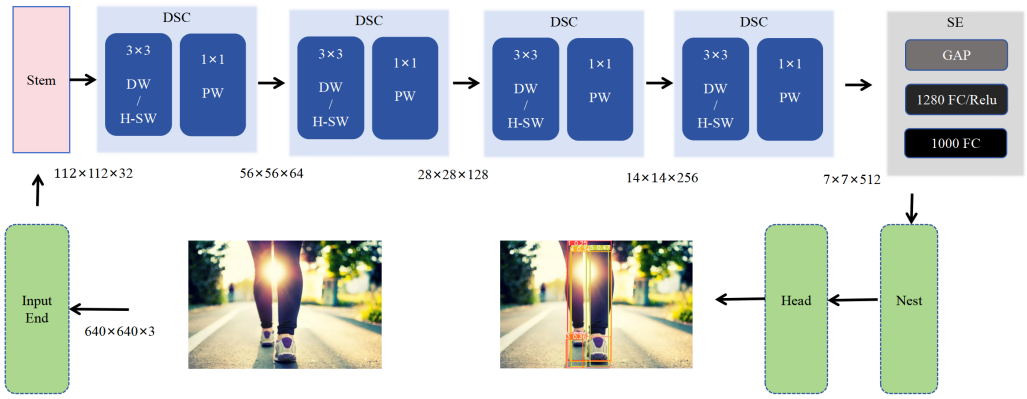


Figure 5: Framework of SLCNet-YOLOv5

4 STUDY DESIGN

In this section, we present the details of our study, including research questions, benchmarks, evaluation metrics, comparison baselines, and experiment environments.

4.1 Research Questions

Dedicated to preventing tracking failures, *it is imperative that the detection model prioritizes speed enhancement within an acceptable range of accuracy loss*, which has been mentioned in Section 1. To assess SLCNet-YOLOv5, we conduct a large-scale study to seek to address the following research questions (RQ). **RQ1**: How does the performance of SLCNet-YOLOv5 compare to baseline networks? **RQ2**: Are there alternative configurations of PP-LCNet that outperform SLCNet? **RQ3**: How is the performance of SLCNet in different scale models? **RQ4**: Will the accuracy loss of the SLCNet-YOLOv5 method decrease significantly? **RQ5** Is a certain level of accuracy loss really acceptable?

4.2 Benchmarks

To evaluate the performance of the SLCNet-YOLOv5 for achieving effective object tracing, we select two representative datasets as benchmarks, including the coco-foot-and-leg and PASCAL VOC. The details of the benchmarks are described as follows.

Coco-foot-and-leg is an adjunct collection derived from the MSCOCO 2017 image dataset, specifically annotated for human body parts. These annotations include categories such as 0 (body), 1 (both feet), 2 (single foot), 3 (both legs), 4 (single leg), and 5 (lower body). The dataset comprises a total of 11,056 images, encompassing the train (9,960), test (984), and validation (112) sets.

PASCAL VOC is a commonly used dataset for object detection and is utilized for further validation of the SLCNet-YOLOv5's generalizability. This dataset encompasses both PASCAL VOC2007 and PASCAL VOC2012, comprising a total of 20 target categories, 16,551 training images, and 4,952 test images.

4.3 Evaluation Metrics

In our evaluation, we employ Average Precision (AP), mean Average Precision (mAP), and Average Detection Processing Time on the CPU end ($ADPT_{CPU}$) to gauge category-specific accuracy, overall accuracy, and detection speed on the CPU edge, respectively.

AP is computed by integrating the precision-recall curve, denoted as

$$AP = \int_0^1 P(R)dR, \tag{2}$$

where P(R) encapsulates the precision-recall relationship, covering all feasible recall values from 0 to 1.

mAP serves as a crucial metric in assessing object detection models. It is derived from the AP calculated for each category detected by the model. The mAP is computed as

$$mAP = \frac{\sum_{i=1}^{N} AP_i}{N}, \tag{3}$$

where N represents the total number of assessed categories, and AP_i denotes the AP for the ith category. This computation involves averaging the AP values across all categories, providing a comprehensive evaluation of the model's object detection accuracy.

ADPT$_{CPU}$ signifies the average time required for object detection per image. To derive this metric, we select 100 images from the test dataset for detection and subsequently compute the average inference time per image.

4.4 Comparison Baselines

This paper introduces a novel lightweight YOLO framework tailored for object detection on CPU platforms. In order to evaluate the efficacy of our approach, we consider four prominent lightweight object detection networks as benchmarks.

CSPDarknet53 is the backbone network of the initial YOLOv5. It introduces the Cross Stage Partial Network (CSP) structure, which can effectively alleviate the gradient vanishing problem and improve training speed and detection accuracy to a certain extent. In this study, we adopt initial YOLOv5 as the state-of-the-art (SOTA) detector.

ShuffleNetv2 [51] is a CNNs architecture optimized for efficient and accurate feature extraction in computer vision tasks, particularly on resource-constrained platforms such as mobile and embedded devices.

MobileNetV2 [37] is a lightweight deep learning CNNs architecture specifically designed for mobile and embedded vision applications. It represents an enhancement over the original MobileNet, aiming to improve performance while maintaining efficiency in terms of computational resources and model size.

MobileNetV3 [18] is the third iteration in the MobileNet series, is engineered to deliver even higher performance and efficiency for mobile and embedded vision tasks. In our study, we replaced the backbone network of YOLOv5s with ShuffleNetv2, MobileNetV2, and MobileNetV3 to create three distinct variants: MobileNetV2-YOLOv5, MobileNetV3-YOLOv5, and ShuffleNetv2-YOLOv5, respectively. These variants serve as the basis for subsequent research endeavors.

4.5 Experiment environments

In order to study the suitability of the trained detection model for lightweight robots, we divided the training and detection processes into two stages and deployed them on different machines.

- In the training process, the experiments were set up with CUDA11.8 and Python3.8 and conducted on the systems equipped with AMD EPYC 7542 @3.20 GHz 32G CPUs and NVIDIA GeForce RTX 4090 24 GB GPUs.
- In the detection process, the experiments run under the Windows operating system and the PyTorch framework. The software environment is CUDA11.8 and Python3.8. The CPU is Intel(R) Core(TM) i7-10750H CPU @ 2.60GHz and the GPU is NVIDIA GeForce RTX 2070 with Max-Q Design.

5 RESULTS AND ANALYSIS

5.1 RQ1: How does the performance of SLCNet-YOLOv5 compare to baseline networks?

Setup and Results. We compute the mAP value at an Intersection over Union (IoU) threshold of 0.5, of SLCNet-YOLOv5 and baseline network in the training end, and then record their $ADPT_{CPU}$ value in the detection end, to indicate the detection accuracy and speed of the models. Additionally, we specify the number of parameters and layers of the trained model to indicate the size of the models. The involved results of SLCNet-YOLOv5 and baselines are shown in Table 1. The bold numbers denote the improvement rate of detection speed and accuracy loss of SLCNet-YOLOv5 compared to the SOTA baseline - native YOLOv5s.

Analysis Our analysis primarily revolves around detection speed on CPU-centric terminal and mAP(0.5) values across SLCNet-YOLOv5 and baselines. 1) In terms of the detection speed, SLCNet-YOLOv5 demonstrates notable efficiency on the CPU-centric detection end, surpassing the SOTA network model. According to Table 1, SLCNet-YOLOv5 achieves respectively 62.8% and 83.1% faster detection speed in two benchmarks compared to YOLOv5s. 2) SLCNet-YOLOv5 showcases the highest detection speed on the detection terminal among other baseline network models. Notably, despite having fewer parameters and layers, MobileNetV2 exhibits the second slowest speed. This disparity may be attributed to a more intricate data flow structure (invisible), that leads to computationally more complex operations and potentially impacts its detection speed in CPU-centric terminal. This highlights the effectiveness of removing activation functions to avoid complex data flow structures in improving speed. 3) Compared with YOLOv5s, the mAP(0.5) value of SLCNet-YOLOv5 decreased by 0.077 and 0.165 for benchmarks, respectively.

Answer to RQ1. Under the requirement in Section 3, SLCNet-YOLOv5 demonstrates superior speed advantage compared to baseline models on two benchmarks, SLCNet-YOLOv5 outperforms the SOTA baseline by up to 62.8% on Coco-foot-and-leg dataset and 83.1% on PASCAL VOC. In terms of detection accuracy, SLCNet-YOLOv5 has a mAP(0.5) loss of 0.077 and 0.165, respectively. The notable enhancements in speed, coupled with a marginal loss in accuracy, indicate that SLCNet-YOLOv5 has good performance to meet the above requirements.

5.2 RQ2: Are there alternative configurations of PP-LCNet that outperform SLCNet?

Setup and Results. In the second research question, we explore four other configurations of PP-LCNet. These configurations involve modifying the activation function during the PW phase, including options such as no activation function, ReLU function, and H-swish function. Additionally, we experiment with altering the size of the convolution kernel to either 3×3 or 5×5 at the network's tail. These different configurations are integrated into the Yolov5s backbone network. The same settings as RQ1 are used for both mAP(0.5) and $ADPT_{CPU}$ calculations. The $ADPT_{CPU}$ and mAP(0.5) results for PP-LCNet (in blue), SLCNet (in red), and four other configurations of PP-LCNet are presented in Table 2. The bold numbers denote the improvement rate of detection speed and accuracy loss of SLCNet compared to the native PP-LCNet.

Analysis As depicted in Table 2, SLCNet exhibits the highest speed across PP-LCNet and other configurations. When compared to the SOTA baseline - PPLCNet, SLCNet surpasses it by up to 12% in Coco-foot-and-leg and 17% in PASCAL VOC in terms of detection speed. Additionally, the mAP(0.5) value of SLCNet decreased by 0.01 and 0.037 for benchmarks.

Answer to RQ2. Under the requirement in Section 3 SLCNet demonstrates superior speed compared to other configurations across both benchmarks. Regarding accuracy, SLCNet exhibits a slight reduction of 0.01 in COCO-foot-and-leg and 0.037 in PASCAL VOC compared to the SOTA baseline. Consequently, SLCNet emerges as the optimal configuration based on PP-LCNet for satisfying the requirement.

5.3 RQ3: How is the performance of SLCNet in different scale models?

Setup and Results. In the third research question, we explore three scales of PP-LCNet in coco-foot-and-leg dataset. The $ADPT_{CPU}$ and mAP(0.5) results for different scales of SLCNet are presented in Table 3. The bold numbers denote the performance metrics of 0.5-scale SLCNet.

Table 1: DETECTION PERFORMANCE OF DIFFERENT BACKBONE NETWORKS ON CPU-CENTRIC END.

Dataset	Coco-foot-and-leg				PASCAL VOC			
Backbone	$ADPT_{CPU}$(ms)	Params	Layers	mAP(0.5)	$ADPT_{CPU}$(ms)	Params	Layers	mAP(0.5)
SLCNet(scale=0.5)	27.3	2996675	294	0.739	27.8	3034433	294	0.602
YOLOv5s	44.8	7026307	157	0.816	50.9	7064065	157	0.767
MobileNet2	45.9	2922819	276	0.791	48.0	2946465	276	0.767
MobileNet3	32.6	2140067	287	0.754	33.9	2159009	287	0.742
Shufflenetv2	35.7	3802211	261	0.786	40.5	3839969	261	0.721
Improvement and Loss	**62.8%** ↑			**0.077**↓	**83.1%**↑			**0.165**↓

Table 2: DETECTION PERFORMANCE OF DIFFERENT CONFIGURATIONS ON CPU-CENTRIC END.

Dataset		Coco-foot-and-leg				PASCAL VOC			
Configurations		$ADPT_{CPU}$(ms)	Params	Layers	mAP(0.5)	$ADPT_{CPU}$(ms)	Params	Layers	mAP(0.5)
-	3*3	**27.3**	2996675	294	0.739	**27.8**	3034433	294	**0.602**
-	5*5	27.9	3013059	294	0.747	28.7	3050817	294	0.648
Relu	3*3	27.7	2996675	320	0.743	28.3	3034433	320	0.595
Relu	5*5	28.1	3013059	320	0.754	29.3	3050817	320	0.643
Hswish	3*3	30.2	2996675	320	0.743	31.2	3034433	320	0.579
Hswish	5*5	30.5	3013059	320	0.749	32.5	3050817	320	0.639
Improvement and Loss		12% ↑			0.01↓	17% ↑			0.037↓

Analysis As depicted in Table 3, compared to the SOTA baseline - YOLOV5s, the speed of larger scale (scale=1) of SLCNet surpasses it by up to 38% in Coco-foot-and-leg. While the accuracy decreased by 0.043, the speed of smaller scale (scale=0.5) of SLCNet surpasses it by up to 82% in Coco-foot-and-leg, While the accuracy decreased by 0.124.

Answer to RQ3. Under the requirement in Section 3 , multi-scale SLCNet are typically robust because they can adapt to targets of different scales, thereby reducing the likelihood of detection failures at specific scales.

5.4 RQ4: Will the accuracy loss of the SLCNet-YOLOv5 method decrease significantly?

Setup and Results. In exploring this research question, we engage in a detailed comparative analysis concerning category-specific accuracy, employing the AP value at an Intersection over Union (IoU) threshold of 0.5. Moreover, the Coco-foot-and-leg dataset is notable for its emphasis on variations in object size, comprising six distinct size categories. In contrast, the PASCAL VOC dataset highlights a wide array of object types, featuring twenty diverse categories. To ensure a thorough evaluation that covers the extensive range of object categories, we rigorously assess the accuracy of each category within these datasets. Figure 6 displays the AP(0.5) results, offering a comparative view of category-specific performance between models trained using SLCNet and baseline networks, as evaluated on the test sets of both benchmark datasets. The polygonal line denotes the accuracy value of SOTA baseline - native YOLOv5s.

Analysis 1) Within the Coco-foot-and-leg dataset, this is evident in the upperside of Figure 6, which depicts the AP(0.5) value of a single-class comparison between SLCNet-YOLOv5 and baseline network models. The findings demonstrate that there is a marginal decrease in accuracy compared to native YOLOv5 for each category regardless of object sizes. 2) In the PASCAL VOC dataset, the accuracy values of SLCNet-YOLOv5's loss similarly fall within a small difference interval, irrespective of the diverse categories.

Table 3: COMPARSION OF DETECTION PERFORMANCE IN DIFFERENT SCALES.

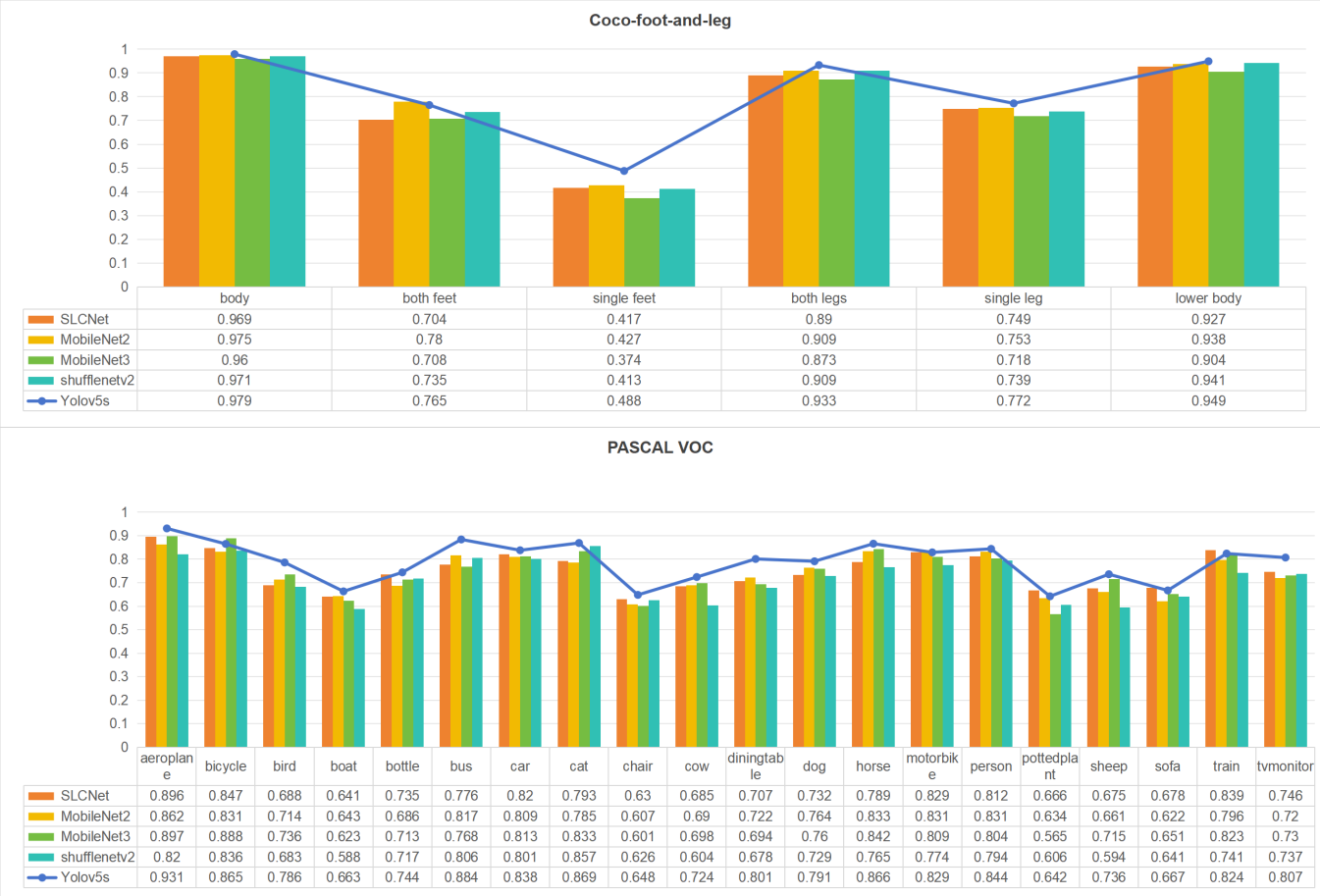
Backbone	Scale	$ADPT_{CPU}$(ms)	mAP(0.5)	Params
YOLOv5s	small	44.8	0.816	7026307
SLCNet	0.25	24.6	0.692	2773723
SLCNet	**0.5**	**27.3**	**0.739**	**2996675**
SLCNet	1	32.4	0.773	3831395

Answer to RQ4. The accuracy values of SLCNet-YOLOv5's loss remain consistently within a narrow difference interval, irrespective of object sizes or across a diverse range of categories.

5.5 RQ5: Is a certain level of accuracy loss really acceptable?

Setup and Results. To ensure the effectiveness of the evaluation, We manually observe and compare categories with relatively high accuracy loss, such as "single foot" and "double feet" categories in Coco-foot-and-leg dataset, "bird", "bus", "diningtable", "horse" and "sheep" categories in PASCAL VOC dataset. We found that the differences in detection accuracy are reflected in the following cases:

- Reduction in confidence values. This case of loss accounts for the vast majority in two datasets. Figure 7 shows the accuracy comparison between SLCNet-YOLOv5 and native YOLOv5s in "single foot" and "double feet" classes, where the box with “2” denotes "double feet" class, a box with "3"



	body	both feet	single feet	both legs	single leg	lower body
SLCNet	0.969	0.704	0.417	0.89	0.749	0.927
MobileNet2	0.975	0.78	0.427	0.909	0.753	0.938
MobileNet3	0.96	0.708	0.374	0.873	0.718	0.904
shufflenetv2	0.971	0.735	0.413	0.909	0.739	0.941
Yolov5s	0.979	0.765	0.488	0.933	0.772	0.949

	aeroplane	bicycle	bird	boat	bottle	bus	car	cat	chair	cow	diningtable	dog	horse	motorbike	person	pottedplant	sheep	sofa	train	tvmonitor
SLCNet	0.896	0.847	0.688	0.641	0.735	0.776	0.82	0.793	0.63	0.685	0.707	0.732	0.789	0.829	0.812	0.666	0.675	0.678	0.839	0.746
MobileNet2	0.862	0.831	0.714	0.643	0.686	0.817	0.809	0.785	0.607	0.69	0.722	0.764	0.833	0.831	0.831	0.634	0.661	0.622	0.796	0.72
MobileNet3	0.897	0.888	0.736	0.623	0.713	0.768	0.813	0.833	0.601	0.698	0.694	0.76	0.842	0.809	0.804	0.565	0.715	0.651	0.823	0.73
shufflenetv2	0.82	0.836	0.683	0.588	0.717	0.806	0.801	0.857	0.626	0.604	0.678	0.729	0.765	0.774	0.794	0.606	0.594	0.641	0.741	0.737
Yolov5s	0.931	0.865	0.786	0.663	0.744	0.884	0.838	0.869	0.648	0.724	0.801	0.791	0.866	0.829	0.844	0.642	0.736	0.667	0.824	0.807

Figure 6: Accuracy of different categories compare to baselines

denotes "single foot" class. Figure 8 b) shows the accuracy comparison between SLCNet-YOLOv5 and native YOLOv5s in "bird" class. The image on the upper is the detection result of YOLOv5s, and the image on the lower is the detection result of this SLCNet-YOLOv5.

- Non-central object lost. This loss occurs in the PASCAL VOC dataset. Figure 8 a) shows the case.
- Additional objects appearing. This loss occurs in the PASCAL VOC dataset. Figure 8 c) shows the case.

Analysis. 1) For the first case of accuracy loss, the findings demonstrate that tiny changes in confidence value, do not affect target tracking tasks. 2) For the second and third cases, whether it is the loss of noncentral objects or the introduction of noncentral objects, as long as the tracking speed is ensured and the tracking target is in the center position, the negative impact of this loss is acceptable.

Answer to RQ5. In target tracking tasks, a marginal decrease in detection accuracy typically does not significantly impact the effectiveness of target tracking in the majority of cases, thus, it is deemed acceptable.

6 RELATED WORK

6.1 Robotic Tracking

Human detection on robots has increasingly gained traction in recent years. This study [9] introduces a resilient approach for human detection implemented on an unmanned ground vehicle (UGV). The approach is grounded in an information fusion strategy that combines data from a laser range finder and images. Both sensors possess distinct strengths and weaknesses. Leveraging them in a complementary manner significantly enhances overall performance compared to using either one in isolation. In recent years, significant progress has been made in human detection applications. Intelligent systems have played a pivotal role in enhancing surveillance and safety in public spaces. Active pedestrian detection systems have become more prevalent in commercial vehicles, and there is a growing trend towards integrating these systems into fully autonomous UAVs [11, 28]. Robots equipped with high computational capabilities typically come with a higher price tag, while conventional lightweight robots lack the capacity to deploy complex large-scale neural networks. Consequently, when robots are tasked with visual object recognition objectives, the adoption of lightweight neural networks becomes imperative.

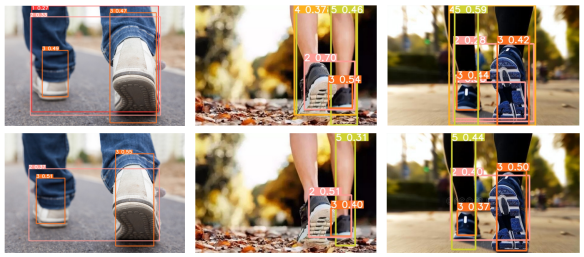


Figure 7: Comparison of detection results on the Coco-foot-and-leg dataset(upside: SLCNet, downside:YOLOv5s)

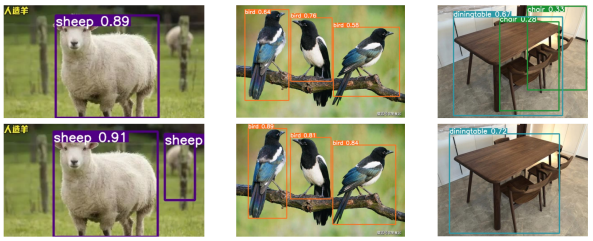


Figure 8: Comparison of detection results on the PASCAL VOC dataset(upside: SLCNet, downside: YOLOv5s)

6.2 CNN-based object detectors

In the realm of academic research, a profusion of techniques and architectural designs has surfaced to address object detection tasks. These encompass a broad spectrum of methodologies, including diverse adaptations of CNNs, such as temporal CNNS [23, 30, 36], three-dimensional CNNs (3D CNNs) [17, 21], and an array of other inventive approaches. CNN-based Object detection can be categorized into one-stage and two-stage methods.

One-stage methods directly predict the location and category of objects in an image, typically offering faster speeds. Examples include YOLO [1, 24, 33–35, 38], FCOS (Fully Convolutional One-Stage Detection) [41, 42, 44, 46, 50], SSD [27], DSSD [10], RetinaNet [25].

Two-stage methods first generate candidate regions and then classify and regress bounding boxes for these regions. Although slower, they often achieve higher accuracy. Common two-stage methods include R-CNN [14], Faster R-CNN[13], Mark R-CNN[15].

6.3 YOLO-based Enhancement

YOLO involves a large amount of advanced researches [3, 7, 12, 19, 22, 31, 32, 39, 45, 49, 52] can be distributed in these components as following.

The backbone network plays a crucial role in extracting high-level features from input images. Recent advancements have focused on replacing traditional CNNs with variants to satisfy diversified needs. EfficientDet [39] has focused on minimizing model parameters and computational requirements while maintaining accuracy. real-time and lightweight object detection tasks. It is characterized by its reduced model size and computational complexity compared to the full YOLOv3 network, making it suitable for deployment on resource-constrained devices such as embedded systems and mobile devices. Tiny-YOLOv3 network[29] reduces model size and computational complexity. ShuffDet [16] improves the YOLOX algorithm by replacing the CSPDark53 model. Gang Liu proposes a lightweight object detection algorithm for robots with an improved YOLOv5 which the C3Ghost and GhostConv modules have been introduced into the YOLOv5 backbone [26].

The neck component serves as an intermediate component that connects the backbone and head, further processing and refining the feature maps. Typically, it includes a Feature Pyramid Network (FPN), which has many variants [12, 19, 22, 32] to address scale diversity and feature fusion challenges in object detection, enhancing the model's perception of targets.

The head component is the top part of the network responsible for performing specific operations for the object detection task. It aims to tackle the diversity and complexity of head structures in object detection[6], often incorporating robust loss functions [3, 31, 52] to improve accuracy.

Our approach is a one-stage detection algorithm and dedicated to tracking tasks on lightweight robots, optimizing algorithms on the CPU side, prioritizing speed improvement while allowing for some degree of accuracy degradation. Our work focuses on distinguishing the major components of backbone networks based on our observation of proper convolutional kernel size and the non-linear representation abilities of activation functions.

7 CONCLUSION AND FUTURE WORK

In this work, we introduce a novel detection framework SLCNet-YOLOv5, tailored specifically for deployment in CPU-centric computing environments on robots. Compared to YOLOv5, the speed has increased by 62.8%, although the accuracy has decreased, but it is within an acceptable range for tracking task. At the same time, the performance of SLCNet-YOLOv5 is also better than other configurations of PP-LCNet. The research results indicate SLCNet-YOLOv5 can serve as a visual recognition network for robot target tracking. We believe that an interesting future direction is to leverage our analyzing techniques to more complex backbones. Replacing several components with CPU-friendly computational blocks is also a promising method for accelerating inference speed.

REFERENCES

[1] Pranav Adarsh, Pratibha Rathi, and Manoj Kumar. 2020. YOLO v3-Tiny: Object Detection and Recognition using one stage improved model. In *2020 6th international conference on advanced computing and communication systems (ICACCS)*. IEEE, 687–694.
[2] AH Adiwahono, VB Saputra, KP Ng, W Gao, Qk Ren, BH Tan, and TW Chang. 2017. Human tracking and following in dynamic environment for service robots. In *TENCON 2017-2017 IEEE Region 10 Conference*. IEEE, 3068–3073.
[3] Kean Chen, Weiyao Lin, Jianguo Li, John See, Ji Wang, and Junni Zou. 2020. AP-loss for accurate one-stage object detection. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 43, 11 (2020), 3782–3798.
[4] Cheng Cui, Tingquan Gao, Shengyu Wei, Yuning Du, Ruoyu Guo, Shuilong Dong, Bin Lu, Ying Zhou, Xueying Lv, Qiwen Liu, et al. 2021. PP-LCNet: A lightweight CPU convolutional neural network. *arXiv preprint arXiv:2109.15099* (2021).
[5] Cheng Cui, Tingquan Gao, Shengyu Wei, Yuning Du, Ruoyu Guo, Shuilong Dong, Bin Lu, Ying Zhou, Xueying Lv, Qiwen Liu, et al. 2021. PP-LCNet: A lightweight CPU convolutional neural network. *arXiv preprint arXiv:2109.15099* (2021).
[6] Xiyang Dai, Yinpeng Chen, Bin Xiao, Dongdong Chen, Mengchen Liu, Lu Yuan, and Lei Zhang. 2021. Dynamic head: Unifying object detection heads with attentions. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 7373–7382.
[7] Xiuli Du, Linkai Song, Yana Lv, and Shaoming Qiu. 2022. A lightweight military target detection algorithm based on improved YOLOv5. *Electronics* 11, 20 (2022),

3263.
[8] Markus Eisenbach, Alexander Vorndran, Sven Sorge, and Horst-Michael Gross. 2015. User recognition for guiding and following people with a mobile robot in a clinical environment. (2015), 3600–3607.
[9] Efstathios P Fotiadis, Mario Garzón, and Antonio Barrientos. 2013. Human detection from a mobile robot using fusion of laser and vision information. *Sensors* 13, 9 (2013), 11603–11635.
[10] Cheng-Yang Fu, Wei Liu, Ananth Ranga, Ambrish Tyagi, and Alexander C Berg. 2017. Dssd: Deconvolutional single shot detector. *arXiv preprint arXiv:1701.06659* (2017).
[11] David Geronimo, Antonio M Lopez, Angel D Sappa, and Thorsten Graf. 2009. Survey of pedestrian detection for advanced driver assistance systems. *IEEE transactions on pattern analysis and machine intelligence* 32, 7 (2009), 1239–1258.
[12] Golnaz Ghiasi, Tsung-Yi Lin, and Quoc V Le. 2019. Nas-fpn: Learning scalable feature pyramid architecture for object detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 7036–7045.
[13] Ross Girshick. 2015. Fast r-cnn. In *Proceedings of the IEEE international conference on computer vision*. 1440–1448.
[14] Ross Girshick, Jeff Donahue, Trevor Darrell, and Jitendra Malik. 2014. Rich feature hierarchies for accurate object detection and semantic segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 580–587.
[15] Kaiming He, Georgia Gkioxari, Piotr Dollár, and Ross Girshick. 2017. Mask r-cnn. In *Proceedings of the IEEE international conference on computer vision*. 2961–2969.
[16] Qiyi He, Ao Xu, Zhiwei Ye, Wen Zhou, and Ting Cai. 2023. Object detection based on lightweight YOLOX for autonomous driving. *Sensors* 23, 17 (2023), 7596.
[17] Rui Hou, Chen Chen, and Mubarak Shah. 2017. An end-to-end 3d convolutional neural network for action detection and segmentation in videos. *arXiv preprint arXiv:1712.01111* (2017).
[18] Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, et al. 2019. Searching for mobilenetv3. In *Proceedings of the IEEE/CVF international conference on computer vision*. 1314–1324.
[19] Miao Hu, Yali Li, Lu Fang, and Shengjin Wang. 2021. A2-FPN: Attention aggregation based feature pyramid network for instance segmentation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 15343–15352.
[20] Md Jahidul Islam, Jungseok Hong, and Junaed Sattar. 2019. Person-following by autonomous robots: A categorical overview. *The International Journal of Robotics Research* 38, 14 (2019), 1581–1618.
[21] Shuiwang Ji, Wei Xu, Ming Yang, and Kai Yu. 2012. 3D convolutional neural networks for human action recognition. *IEEE transactions on pattern analysis and machine intelligence* 35, 1 (2012), 221–231.
[22] Alexander Kirillov, Ross Girshick, Kaiming He, and Piotr Dollár. 2019. Panoptic feature pyramid networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 6399–6408.
[23] Colin Lea, Michael D Flynn, Rene Vidal, Austin Reiter, and Gregory D Hager. 2017. Temporal convolutional networks for action segmentation and detection. In *proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 156–165.
[24] Hao Li, Lianbing Deng, Cheng Yang, Jianbo Liu, and Zhaoquan Gu. 2021. Enhanced YOLO v3 tiny network for real-time ship detection from visual image. *Ieee Access* 9 (2021), 16692–16706.
[25] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. 2017. Focal loss for dense object detection. In *Proceedings of the IEEE international conference on computer vision*. 2980–2988.
[26] Gang Liu, Yanxin Hu, Zhiyu Chen, Jianwei Guo, and Peng Ni. 2023. Lightweight object detection algorithm for robots with improved YOLOv5. *Engineering Applications of Artificial Intelligence* 123 (2023), 106217.
[27] Wei Liu, Dragomir Anguelov, Dumitru Erhan, Christian Szegedy, Scott Reed, Cheng-Yang Fu, and Alexander C Berg. 2016. Ssd: Single shot multibox detector. In *Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part I 14*. Springer, 21–37.
[28] Eleftherios Lygouras, Nicholas Santavas, Anastasios Taitzoglou, Konstantinos Tarchanidis, Athanasios Mitropoulos, and Antonios Gasteratos. 2019. Unsupervised human detection with an embedded vision system on a fully autonomous UAV for search and rescue operations. *Sensors* 19, 16 (2019), 3542.
[29] Samah AF Manssor, Shaoyuan Sun, Mohammed Abdalmajed, and Shima Ali. 2022. Real-time human detection in thermal infrared imaging at night using enhanced Tiny-yolov3 network. *Journal of Real-Time Image Processing* (2022), 1–14.
[30] Nitin Nair, Chinchu Thomas, and Dinesh Babu Jayagopi. 2018. Human activity recognition using temporal convolutional network. In *Proceedings of the 5th international Workshop on Sensor-based Activity Recognition and Interaction*. 1–8.
[31] Kemal Oksuz, Baris Can Cam, Emre Akbas, and Sinan Kalkan. 2020. A ranking-based, balanced loss function unifying classification and localisation in object detection. *Advances in Neural Information Processing Systems* 33 (2020), 15534–15545.
[32] Siyuan Qiao, Liang-Chieh Chen, and Alan Yuille. 2021. Detectors: Detecting objects with recursive feature pyramid and switchable atrous convolution. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 10213–10224.
[33] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. 2016. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 779–788.
[34] Joseph Redmon and Ali Farhadi. 2017. YOLO9000: better, faster, stronger. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 7263–7271.
[35] Joseph Redmon and Ali Farhadi. 2018. Yolov3: An incremental improvement. *arXiv preprint arXiv:1804.02767* (2018).
[36] ELizabeth Sabu and K Suresh. 2018. Object detection from video using temporal convolutional network. In *2018 IEEE Recent Advances in Intelligent Computational Systems (RAICS)*. IEEE, 11–15.
[37] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. 2018. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 4510–4520.
[38] Yuheng Shi, Naiyan Wang, and Xiaojie Guo. 2023. YOLOV: Making still image object detectors great at video object detection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 37. 2254–2262.
[39] Mingxing Tan, Ruoming Pang, and Quoc V Le. 2020. Efficientdet: Scalable and efficient object detection. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 10781–10790.
[40] Ryosuke Tasaki, Michiteru Kitazaki, Jun Miura, and Kazuhiko Terashima. 2015. Prototype design of medical round supporting robot "Terapio". (2015), 829–834.
[41] Zhi Tian, Xiangxiang Chu, Xiaoming Wang, Xiaolin Wei, and Chunhua Shen. 2022. Fully convolutional one-stage 3d object detection on lidar range images. *Advances in Neural Information Processing Systems* 35 (2022), 34899–34911.
[42] Z Tian, C Shen, H Chen, and T He. 2019. FCOS: Fully Convolutional One-Stage Object Detection. arXiv. *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)* (2019), 9627—-9636.
[43] Chien-Yao Wang, Alexey Bochkovskiy, and Hong-Yuan Mark Liao. 2023. YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 7464–7475.
[44] Jianfeng Wang, Lin Song, Zeming Li, Hongbin Sun, Jian Sun, and Nanning Zheng. 2021. End-to-end object detection with fully convolutional network. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 15849–15858.
[45] Rongsheng Wang, Yaofei Duan, Menghan Hu, Xiaohong Liu, Yukun Li, Qinquan Gao, Tong Tong, and Tao Tan. 2023. LightR-YOLOv5: A compact rotating detector for SARS-CoV-2 antigen-detection rapid diagnostic test results. *Displays* 78 (2023), 102403.
[46] Tai Wang, Xinge Zhu, Jiangmiao Pang, and Dahua Lin. 2021. Fcos3d: Fully convolutional one-stage monocular 3d object detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 913–922.
[47] Wentong Wu, Han Liu, Lingling Li, Yilin Long, Xiaodong Wang, Zhuohua Wang, Jinglun Li, and Yi Chang. 2021. Application of local fully Convolutional Neural Network combined with YOLO v5 algorithm in small target detection of remote sensing image. *PloS one* 16, 10 (2021), e0259283.
[48] Ruibin Xiong, Yunchang Yang, Di He, Kai Zheng, Shuxin Zheng, Chen Xing, Huishuai Zhang, Yanyan Lan, Liwei Wang, and Tieyan Liu. 2020. On layer normalization in the transformer architecture. In *International Conference on Machine Learning*. PMLR, 10524–10533.
[49] Hao Xu, Bo Li, and Fei Zhong. 2022. Light-YOLOv5: A lightweight algorithm for improved YOLOv5 in complex fire scenarios. *Applied Sciences* 12, 23 (2022), 12312.
[50] Jiexian Zeng, Jiale Xiong, Xiang Fu, and Lu Leng. 2020. ReFPN-FCOS: One-stage object detection for feature learning and accurate localization. *IEEE Access* 8 (2020), 225052–225063.
[51] Xiangyu Zhang, Xinyu Zhou, Mengxiao Lin, and Jian Sun. 2018. Shufflenet: An extremely efficient convolutional neural network for mobile devices. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 6848–6856.
[52] Zhaohui Zheng, Ping Wang, Wei Liu, Jinze Li, Rongguang Ye, and Dongwei Ren. 2020. Distance-IoU loss: Faster and better learning for bounding box regression. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 34. 12993–13000.